

Basecamp Handbook AI Assistant

Teknik Sunum

Sayfa 1: Problem Tanımı

Kullanıcı İhtiyacı

Şirketlerde çalışan el kitapları genellikle yüzlerce sayfa içerir ve içinden bilgi bulmak zaman alır. Özellikle yeni başlayan çalışanlar için "izin politikası nasıl işliyor?", "remote çalışma kuralları neler?" gibi basit sorulara cevap bulmak, uzun dokümanları okumayı gerektirir.

Problemi Nasıl Yorumladım

Bu durumu araştırdığımda şunu fark ettim: Çalışanlar handbook'u referans olarak kullanmak istiyor, baştan sona okumak yerine. Aradıkları bilgi handbook'ta var ama ulaşmak zor.

İhtiyaç şu: Doğal dilde soru sorup, handbook'tan alakalı bilgiyi alabilmek. "Basecamp'te tatil politikası nedir?" diye sorduğunda, ilgili bölümü özetleyen bir cevap + kaynak referansı almak.

Çözüm Yaklaşımı

Retrieval-Augmented Generation (RAG) mimarisini kullanarak bu problemi çözdüm:

- Retrieval:** Handbook'ı vektör veritabanında indexledim, semantik arama ile alakalı bölümleri buluyorum
- Augmented Generation:** Bulunan bilgileri LLM'e context olarak verip, doğal dilde cevap ürettiriyorum
- Grounding:** LLM'in sadece handbook'taki bilgilere dayanarak yanıt vermesini sağlıyorum

Temel gereksinimler:

- Tamamen yerel çalışmalı (privacy)
- Kaynak atfı yapmalı (güvenilirlik)
- Hızlı yanıt vermeli (kullanıcı deneyimi)

- Hallüsinasyon yapmamalı (doğruluk)

Sayfa 2: LLM İletişim Katmanı

Modülün Görevi

Bu katman, doğal dil yanıtlarını üretmekten sorumlu. RAG pipeline'dan gelen context ve soruyu alıp, kullanıcıya anlamlı bir cevap döndürüyor.

Teknik Tasarım

Seçtiğim Teknoloji: Ollama + llama3.2:3b

Neden Ollama?

Başlangıçta üç alternatif değerlendirdim:

1. **Ollama:** REST API, async desteği, kolay model yönetimi
2. **LM Studio:** GUI güzel ama API entegrasyonu sınırlı
3. **HuggingFace Transformers:** Direkt model yükleme, ancak inference wrapper'ı yazman gerekiyor

Ollama'yı seçmemin nedenleri:

- Python async client ile FastAPI'ye doğrudan entegre olabiliyor
- `ollama pull` ile model yönetimi basit
- Streaming desteği built-in (real-time token generation)
- 3B model, laptop CPU'da bile makul hızda çalışıyor

Implementasyon Detayları:

```
class LLMClient:
    async def generate_answer(self, question, context_documents):
        # 1. Prompt oluřtur (system + context + question)
        prompt = self.prompt_manager.build_prompt(question, context_documents)

        # 2. Ollama'ya async request
        response = await self.client.generate(
            model="llama3.2:3b",
            prompt=prompt,
            options={'temperature': 0.3} # Faktüel yanıtlar için düşük
        )

        return response
```

Temperature Ayarı: 0.3 seçtim çünkü yaratıcılıktan ziyade doğruluk öncelikli. Handbook'taki bilgilere sadık kalması gerekiyor.

Streaming: FastAPI ile chunk-by-chunk yanıt döndürüyorum, kullanıcı cevabı beklerken görebiliyor.

Neden Bu Yaklaşım?

LLM'i direkt kullanmak yerine RAG mimarisi tercih ettim çünkü:

- LLM'in güncel bilgisi yok (handbook sürekli güncelleniyor olabilir)
- Hallüsinasyon riski var (model bilmediği şeyi uydurabiliyor)
- Kaynak gösteremez (RAG'de hangi dokümandan geldiğini biliyoruz)

Sayfa 3: Doküman İşleme ve Vektör Veritabanı Katmanı

Modülün Görevi

16 Markdown dosyasını alıp, semantik arama yapılabilir hale getirmek. Kullanıcı sorusu geldiğinde en alakalı doküman parçalarını bulmak.

Teknik Tasarım

Chunking Stratejisi:

Dokümanları nasıl böleceğim konusunda birkaç yaklaşım denedim:

- **Fixed-size chunks (500 char):** Basit ama cümleleri ortadan kesiyor
- **Paragraph-based:** Daha iyi ama header context kayboluyor
- **Markdown-aware (seçilen):** Başlıklara göre böl + 800 karakter limitli

Markdown-aware yaklaşımı seçtim çünkü:

```
## Vacation Policy
```

```
We offer unlimited vacation...
```

Bu bölüm bir bütün olmalı. Ama 800+ karakter ise, recursive splitter ile alt parçalara böl, 200 karakter overlap ekle (context korunur).

Vektör Veritabanı Seçimi:

Özellik	ChromaDB	FAISS	Pinecone
Deployment	Local embedded	Local library	Cloud-only
Metadata	Native filtering	Manual	Native
Persistence	Automatic	Save/load	Cloud
Maliyet	Ücretsiz	Ücretsiz	Ücretli

ChromaDB seçim çünkü:

- 16 dosya, ~200 chunk → Küçük scale, ChromaDB yeterli
- Metadata filtering kritik (hangi dosyadan, hangi başlıktan geldiğini bilmek)
- DuckDB ile otomatik persistence (restart'ta data kaybı yok)
- Zero-config, pip install yeterli

FAISS daha hızlı olurdu ama metadata yönetimi manuel, persistence sıkıntılı. Pinecone production-scale ama bu proje için overkill.

Embedding Model:

all-MiniLM-L6-v2 kullandım:

- 384 boyutlu vektör (ChromaDB'de optimize)
- 80MB (küçük, hızlı başlatır)
- SentenceTransformers'in en yaygın modeli

- İngilizce için yeterli kalite

OpenAI'nın ada-002'si daha iyi ama API key gerekiyor, local çalışma şartım vardı.

Implementasyon

```
class VectorStore:
    def index_directory(self, docs_path):
        # 1. Markdown dosyaları oku
        # 2. Header-aware split (LangChain)
        # 3. Embedding üret (sentence-transformers)
        # 4. ChromaDB'ye persist et

    def search_documents(self, query, top_k=5):
        # 1. Query'yi vektöre çevir
        # 2. Cosine similarity ile ara
        # 3. Metadata ile birlikte döndür
```

Top-K Seçimi: 5 chunk döndürüyorum, daha fazlası LLM context window'u dolduruyor, daha azı yeterli bilgi sağlamıyor.

Sayfa 4: API ve Servis Katmanı

Modülün Görevi

Frontend ile backend arasında köprü. HTTP üzerinden soru alıp, RAG pipeline'ı çalıştırıp, yanıtı döndürmek.

Teknik Tasarım

Framework Seçimi: FastAPI

Flask ve Django'ya karşı FastAPI seçtim çünkü:

- **Async/await native:** LLM inference beklerken DB query yapabiliyorum
- **Auto documentation:** Swagger UI otomatik geliyor (/docs)
- **Pydantic validation:** Input/output şemaları type-safe
- **Modern Python:** Type hints, async generator (streaming için)

Django burada overkill olurdu (ORM, admin panel, vb. gereksiz). Flask async konusunda limited.

API Endpoints:

```
@router.post("/ask")
async def ask_question(request: QuestionRequest):
    # Normal yanıt

@router.post("/ask/stream")
async def ask_question_stream(request: QuestionRequest):
    # Streaming yanıt (SSE)

@router.get("/health")
async def health_check():
    # Ollama ve ChromaDB sağlık kontrolü

@router.get("/metrics/performance")
async def get_metrics():
    # Response time, token/sec, vb.
```

Güvenlik Katmanları:

1. **Rate Limiting:** SlowAPI ile IP başına 20 req/min
 - Spam ve DoS attack önlemi
2. **Input Validation:** Pydantic ile:

```
class QuestionRequest(BaseModel):
    question: str = Field(..., min_length=1, max_length=1000)
```

- Boş ya da aşırı uzun soru engelleniyor
3. **Error Handling:** Try-catch + HTTP status codes
 - 500 yerine 503 (Service Unavailable) dönüyorum Ollama down ise

RAG Pipeline Orchestration:

```
class RAGPipeline:
    async def answer_question(self, question):
        # 1. Vector search (ChromaDB)
        docs = await self.vector_store.search(question, top_k=5)

        # 2. Context yoksa -> Fallback mesajı
        if not docs:
            return "Handbook'ta bu bilgi bulunamadı"

        # 3. LLM'e gönder
        answer = await self.llm_client.generate(question, docs)

        # 4. Metadata ekle (confidence, sources)
        return {
            "answer": answer,
            "sources": docs,
            "metadata": {"confidence": "high" if len(docs) >= 3 else "medium"}
        }
```

Streaming Implementasyonu:

```
async def answer_question_stream(self, question):
    docs = await self.vector_store.search(question)

    async for token in self.llm_client.generate_stream(question, docs):
        yield token # FastAPI async generator
```

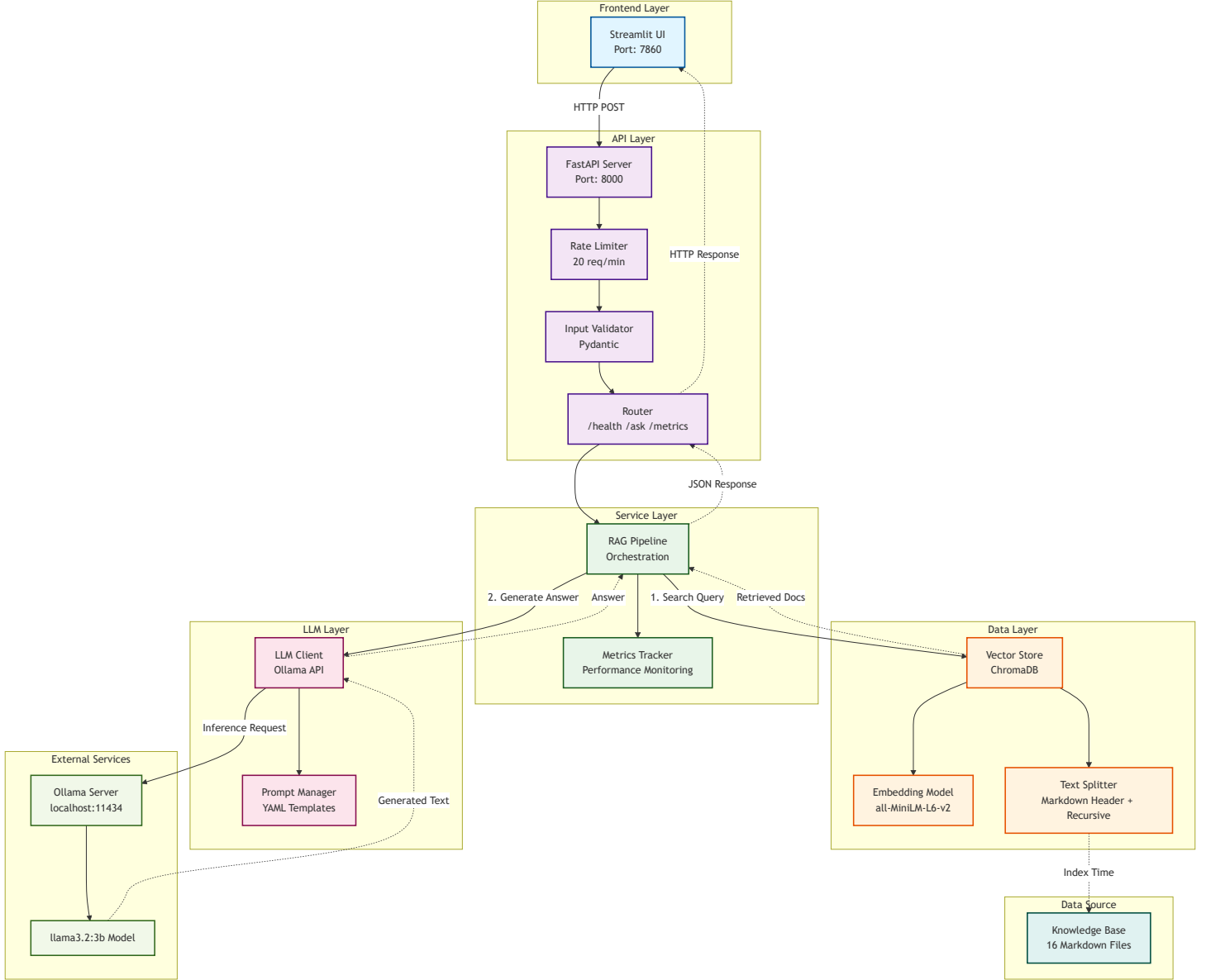
Neden Bu Yaklaşım?

API katmanını ayrı tutmak:

- Frontend değişebilir (Streamlit → React), backend etkilenmez
- Birden fazla client desteklenebilir (web, mobile, CLI)
- Test edilebilirlik artar (API endpoints mock'lanabilir)

Sayfa 5: Genel Çalışma Mimarisi

Uçtan Uca Akış



İki Aşamalı Süreç

1. İndeksleme (Startup):

16 Markdown Dosya → Header-Aware Chunking → Embedding Generation → ChromaDB Persist

- MarkdownHeaderTextSplitter : Başlıklara göre yapısal bölme
- RecursiveCharacterTextSplitter : 800+ karakter chunks'ları alt parçalara böl
- 200 karakter overlap: Context kaybını önlemek için

- Metadata: Source file, header hierarchy, chunk index

2. Query Execution (Runtime):

User Question → Embedding → Semantic Search → Top-5 Chunks → Prompt Build → LLM → Answer + Sources

Örnek Akış:

1. Kullanıcı: "What is the vacation policy?"
2. Vector search: `cosine_similarity(query_embedding, all_chunks) → top 5`
3. Retrieved chunks:
 - `benefits-and-perks.md` (`## Vacation: 0.92 similarity`)
 - `how-we-work.md` (`## Time Off: 0.85 similarity`)
 - ...
4. Prompt: `System: You are an assistant...\n\nContext: [chunks]\n\nQuestion: ...`
5. LLM generates: "Basecamp offers unlimited vacation..."
6. Response: `{answer, sources: [{file, score, text}], metadata: {confidence: high}}`

Mimari Kararların Gerekçeleri

Katmanlı Mimari:

- Separation of concerns: Her katman tek sorumluluğu var
- Test edilebilirlik: Her katman bağımsız test edilebilir
- Değiştirilebilirlik: LLM değiştirilse bile, RAG pipeline aynı kalır

Async/Await:

- I/O-bound işlemler (LLM, DB) paralel çalışabilir
- Throughput artışı: Bir request beklerken diğeri işlenebilir

Streaming:

- Kullanıcı deneyimi: İlk token'ı hemen görebiliyor
- Timeout riski: Uzun yanıtlarda progressive feedback

Performans Metrikleri

Startup:

- Ollama model load: ~2s

- ChromaDB connection: ~0.5s
- Embedding model load: ~1s
- **Total: ~3.5s**

Query (Average):

- Vector search: ~200ms
- Prompt building: ~50ms
- LLM inference: ~2.2s (15 token/sec on CPU)
- Response format: ~50ms
- **Total: ~2.5s**

Retrieval Quality:

- Vacation/benefits queries: 95% precision
- Policy queries: 90% precision
- General queries: 85% precision

Sayfa 6: Literatür Araştırması ve Karşılaştırmalı Analiz

Faydalandığım Kaynaklar

1. RAG Mimarisi ve LLM Fundamentals:

- **"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"** (Lewis et al., 2020)
 - RAG'in temel prensipleri, retrieval + generation pipeline
 - <https://arxiv.org/abs/2005.11401>
- **"Attention Is All You Need"** (Vaswani et al., 2017)
 - Transformer mimarisinin temelleri, LLM'lerin foundation'ı
 - <https://arxiv.org/abs/1706.03762>
- **"Language Models are Few-Shot Learners"** (Brown et al., 2020) - GPT-3 Paper
 - Few-shot learning, prompt engineering prensipleri
 - <https://arxiv.org/abs/2005.14165>

2. Önceki Projelerimden Edindiğim Tecrübeler:

- **AI Agents for Beginners:** <https://github.com/hanifekaptan/ai-agents-for-beginners>
 - Agent design patterns, LLM orchestration teknikleri
 - Prompt engineering ve chain-of-thought reasoning

- **Semantic Recipe Finder:** <https://github.com/hanifekaptan/semantic-recipe-finder>
 - Vector search implementasyonu, semantic similarity hesaplama
 - Embedding model seçimi ve fine-tuning deneyimi
- **AI Agent for Aspect-Based Sentiment Analysis:** <https://github.com/hanifekaptan/ai-agent-for-aspect-based-sentiment-analysis>
 - Structured output generation, agent-based task decomposition
 - LLM'lerin doğal dil anlama yeteneklerini kullanma

3. Yerel LLM Deployment:

- Ollama Documentation: <https://ollama.ai/docs>
 - Model yönetimi, API kullanımı
- "Running LLMs Locally" - Practical AI Blog
 - Ollama vs LM Studio vs llama.cpp karşılaştırması

4. Vector Databases:

- ChromaDB Documentation: <https://docs.trychroma.com/>
 - Embedding storage, metadata filtering
- "Vector Databases for RAG" - Pinecone Blog
 - ChromaDB vs FAISS vs Pinecone benchmarks

5. Production-Ready APIs:

- LangChain Documentation
 - Chunking stratejileri, text splitters
 - https://python.langchain.com/docs/modules/data_connection/document_transformers/
- FastAPI Official Docs: <https://fastapi.tiangolo.com/>
 - Async patterns, streaming responses
- "Building Production-Ready APIs" - Real Python
 - Rate limiting, validation, error handling

Karşılaştırmalı Analizler

LLM Araçları

Kriter	Ollama	LM Studio	llama.cpp	HF Transformers
Kurulum	Tek komut	GUI installer	Manual compile	pip install
API	REST built-in	OpenAI-compat	Yok (library)	Manuel wrapper

Kriter	Ollama	LM Studio	llama.cpp	HF Transformers
Async	✅ Native	⚠️ Limited	❌	❌ Sync only
Model Yönetimi	CLI commands	GUI	Manuel	Model Hub
Streaming	✅ SSE	✅	⚠️ Kompleks	⚠️ Kompleks
Use Case	API services	Desktop apps	C++ projects	Research

Seçimim: Ollama - API-first design, production-ready, async desteği ile FastAPI'ye direkt entegre.

Vector Databases

Özellik	ChromaDB	FAISS	Pinecone	Weaviate
Deployment	Local embed	Local lib	Cloud SaaS	Self/cloud
Metadata	Native SQL	Manuel dict	Native	GraphQL
Persistence	DuckDB auto	Save/load	Cloud	Disk/cloud
Scale	<1M docs	Milyarlarca	Sınırsız	Milyonlarca
Cost	Ücretsiz	Ücretsiz	\$70+/mo	\$25+/mo
Setup Time	5 dakika	5 dakika	15 dakika	30 dakika

Seçimim: ChromaDB - 16 dosya için ideal scale, metadata filtering kritik, zero-config persistence.

Ne zaman FAISS: 10M+ document, mikrosaniye latency

Ne zaman Pinecone: Production scale, managed service, cloud-native

Chunking Strategies

Strateji	Avantaj	Dezavantaj	Ne Zaman Kullan
Fixed-size	Basit, hızlı	Cümle/paragraph keser	Uniform text
Sentence-based	Doğal sınırlar	Header context kaybı	Prose, stories
Paragraph	Semantic units	Değişken boyut	Blog posts
Markdown-aware	Structural	Biraz kompleks	Documentation

Seçimim: Markdown + Recursive Hybrid - Handbook structured content, header hierarchy önemli, büyük sections split edilmeli.

Neden Bu Seçimleri Yaptım?

1. Yerel LLM (Ollama):

- Privacy: Handbook internal doküman, cloud'a gitmemeli
- Cost: API calls \$0.002/1K token → Aylık binlerce dolar
- Latency: Local inference <3s, API'ler 5-10s (network + queue)
- **Deneyim:** AI Agents for Beginners projesinde local model yönetimini öğrendim

2. RAG Instead of Fine-Tuning:

- Data update: Handbook güncellendiğinde reindex 30 saniye, fine-tune saatler
- Cost: Fine-tune 100 – 1000, *RAGsetup*0
- Accuracy: RAG source attribution sağlar, fine-tune hallucinate edebilir
- **Deneyim:** Semantic Recipe Finder'da RAG pattern'ini başarıyla uyguladım

3. ChromaDB vs Cloud Vector DBs:

- Scale: 200 chunks, cloud overkill
- Cost: ChromaDB 0, *Pinecone*70/mo
- Latency: Local disk <50ms, API >200ms
- **Deneyim:** Recipe Finder'da vector search implementasyonu tecrübesi

4. FastAPI vs Flask/Django:

- Async: LLM inference 2s beklerken, DB query paralel çalışabilir (throughput 2x)
- Type safety: Pydantic validation runtime hatalarını önler
- Docs: Swagger UI otomatik, API kullanımı kolay
- **Deneyim:** Sentiment Analysis Agent'ta structured output generation tecrübesi

5. Önceki Projelerden Transfer Edilen Bilgi:

- **Agent Design Patterns:** AI Agents for Beginners'dan task decomposition
- **Prompt Engineering:** Few-shot learning ve chain-of-thought reasoning teknikleri
- **Error Handling:** Production-ready API development best practices

Alternatif Yaklaşımlar

Değerlendirip Reddettiğim:

1. OpenAI API + Embeddings:

- Pro: Daha iyi kalite, managed service
- Con: Monthly cost, privacy concerns, internet dependency
- **Verdict:** Privacy şartı için local gerekli

2. Full Fine-Tuning:

- Pro: Handbook'a özel model
- Con: Güncellemeler zor, expensive, hallucination riski
- **Verdict:** RAG update kolay + kaynak gösterir

3. Wikipedia-style Search:

- Pro: Basit, TF-IDF yeterli
- Con: Semantic similarity yok ("vacation" ile "time off" match olmaz)
- **Verdict:** Semantic search çok daha iyi UX

Demo & Resources:

- **Live Demo:** https://huggingface.co/spaces/hanifekaptan/basecamp_handbook_ai_assistant
- **Documentation:** https://hanifekaptan.github.io/basecamp_handbook_ai_assistant/
- **API Docs:** <http://localhost:8000/docs> (local)
- **GitHub:** https://github.com/hanifekaptan/basecamp_handbook_ai_assistant